Tool Calling Under Interface Drift: A Non-Oracle Study of Validation and Repair

Anonymous Authors

Anonymous Submission

Abstract. Tool-calling language models depend on surface-form tool interfaces (descriptions, schemas, and tool lists) that can drift after deployment even when tool semantics do not. We study this problem as *interface drift* and evaluate whether a lightweight, training-free inference-time repair layer can recover reliability without retraining. Our method, SchemaShield-Lite, combines schema validation with a candidate-set retry when tool names are missing and a one-shot constrained repair call. On our primary model (Qwen3.5-9B), it raises DICE-300 accuracy from 0.717 to 0.813 and BFCL-200 from 0.580 to 0.805, recovering 19/22 and 33/45 drift-induced failures with no observed repair harms. A naive-retry baseline does not explain the gains. Ablation reveals that candidate-set drift (adding distractor tools) accounts for nearly all observed damage on our DICE slice, while description and schema edits alone cause negligible degradation. In the non-oracle setting, missing tool names emerge as the main repair bottleneck. A same-protocol BFCL replication on Qwen3.5-35B-A3B (0.635 → 0.845) supports the effect within the Qwen family, though cross-family portability remains limited.

Keywords: tool calling, language models, robustness, interface drift, repair

1 Introduction

Modern language-model agents increasingly depend on external tools for retrieval, execution, planning, and actuation. In practice, however, tool use is mediated entirely through text: tool descriptions, parameter schemas, enum values, and candidate tool lists. Recent work shows that these interface details strongly affect tool selection and argument generation even when the tool semantics are unchanged [3,1]. This creates a deployment problem: tool interfaces drift over time as developers rename parameters, revise descriptions, add examples, or expand candidate tool sets.

We study this problem as *interface drift*: benign, semantics-preserving changes to tool interfaces that occur after deployment. Our goal is not to train a new tool-calling model, but to quantify drift sensitivity and test whether a compact inference-time repair layer can recover reliability without retraining.

We instantiate drift with three deployment-motivated synthetic perturbations: (i) stale description examples that mention legacy field names, (ii) schema

renaming, and (iii) distractor tools. Our primary evaluation uses Qwen3.5-9B on a 300-example DICE-BENCH slice [4] and a 200-example BFCL slice [20] under a deployment-realistic non-oracle repair protocol. Earlier oracle-constrained sweeps on additional open models are retained only as upper-bound context and for secondary analyses.

This paper makes three empirical contributions, demonstrated primarily on Qwen3.5-9B with a supporting replication on Qwen3.5-35B-A3B:

1. **Non-oracle inference-time repair recovers drift-induced failures on the primary model.** On Qwen3.5-9B, SchemaShield-Lite raises DICE-300 accuracy from 0.717 to 0.813 and BFCL-200 accuracy from 0.580 to 0.805, while showing no observed repair harms on the originally-correct slice in either benchmark. A same-protocol BFCL replication on Qwen3.5-35B-A3B ($0.635 \rightarrow 0.845$) confirms the effect is not confined to a single model size, though the cleanest non-oracle evidence remains within the Qwen family.
2. **Naive retry does not explain the gains.** In the separate baseline runs reported in Section 6.2, naive retry reaches 0.727 versus repaired 0.813 on DICE-300 and 0.610 versus 0.795 on BFCL-200. A second attempt alone therefore cannot explain the recovery.
3. **Missing tool names are the main non-oracle bottleneck, and most primary-model drift misses remain validator-visible.** In the non-oracle setting, adding a candidate-set retry improves repaired accuracy from 0.780 to 0.813 on DICE and from 0.725 to 0.805 on BFCL while reducing unresolved repair targets from 35 to 15 and from 34 to 9 respectively. In the final primary runs, only 2/22 drift misses on the DICE originally-correct slice and 0/45 on BFCL remain validator-invisible.

2 Related Work

Recent work has established that tool interfaces are not neutral wrappers around tool use; they are part of the learning and inference problem itself. Guo et al. [3] show that rewriting tool descriptions can substantially improve agent reliability, especially when candidate tool sets are large. Faghih et al. [1] show the opposite direction: small edits to tool descriptions can dramatically shift tool preference, exposing a fragility in description-mediated tool use. Rabinovich and Anaby-Tavor [13] study robustness in agentic function calling under toolkit changes and other perturbations, while our focus is narrower: post-deployment interface drift with fixed underlying tool semantics.

Several recent papers study robustness and recovery in tool-use systems. Zhang et al. [23] show that execution-error recovery remains a major weakness, particularly for smaller models, and propose RL-based training to improve it. Our setting is narrower and more deployment-oriented: we target inference-time robustness under interface drift rather than post-training optimization.

Our repair stack also connects to structured-output enforcement via constrained decoding. Grammar-constrained and grammar-aligned decoding study

how to guarantee or preserve valid structured outputs during generation [2,10]. We instead study post-hoc validation and repair after the base model has already produced a drifted tool call.

Benchmark work has also highlighted that realistic function calling is still far from solved. ComplexFuncBench introduces multi-step and constrained long-context function calling [24], while DICE-BENCH focuses on multi-round, multi-party dialogues with tool-related information dispersed across conversation turns [4]. Other benchmarks, including API-Bank, τ-bench, and ToolSandbox, similarly stress planning, state, and interaction in tool-use settings [6,21,8]. We evaluate on both DICE-BENCH and the Berkeley Function Calling Leaderboard (BFCL) [20] because they capture complementary settings: DICE-BENCH exercises conversational multi-turn dialogues, while BFCL tests single-turn tool calls across diverse APIs.

Broader tool-use work has focused on teaching models to call APIs, retrieving relevant tools, and combining reasoning with action [14,11,12,22]. Our focus is different: robustness when an already-deployed tool interface changes.

This setting also connects to software-engineering work on API evolution and contract checking. AutoGuard, for example, compares OpenAPI descriptions across versions and flags changes to methods, parameters, responses, and schemas [5]. Serbout and Pautasso [15] study how Web API versioning practices align with the actual changes made across releases. We borrow the intuition that interface evolution is a first-class engineering concern, but study its effect on LLM tool use rather than on conventional API clients.

Compared with dataset-generation work such as APIGen [7], our objective is different. We do not create new large-scale tool-calling training data. Instead, we evaluate whether a lightweight repair layer can stabilize existing models against a specific class of post-deployment failures.

3 Problem Formulation

Let a tool be represented by an interface $I = (d, s, C)$, where d is the natural-language description, s is the argument schema, and C is the candidate tool set shown to the model. The underlying tool semantics are assumed fixed. An *interface drift* operator Δ maps I to a new interface $\Delta(I)$ that preserves tool semantics but perturbs surface form. For clarity, we view Δ as a composition of three operators,

$$\Delta = \delta_c \circ \delta_s \circ \delta_d,$$

where δ_d edits only the description text d, δ_s edits only the schema surface form s, and δ_c edits only the visible candidate set C. Each operator is semantics-preserving with respect to the benchmark gold tool call; the evaluation question is whether the model remains robust to these surface-form changes at inference time.

We study three drift types:

- **Description drift**: add stale or legacy examples to the description that refer to older field names.

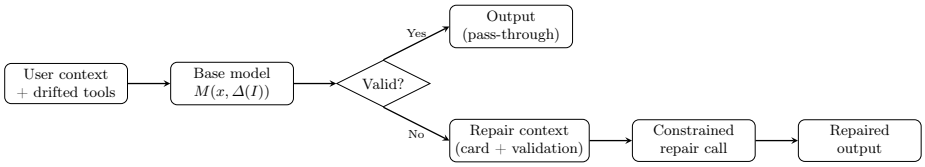

Fig. 1: SchemaShield-Lite pipeline. Invalid tool calls trigger structured validation and constrained repair; missing tool names are first resolved via candidate-set retry.

- **Schema drift**: rename parameters while preserving semantics.
- **Candidate-set drift**: add distractor tools with overlapping descriptions.

Given user context x, tool interface I, and model M, the base system outputs a tool call $y = M(x, I)$. We evaluate three stages:

1. **Original**: inference on the original tool interface.
2. **Drifted**: inference on the drifted interface $\Delta(I)$.
3. **Repaired**: inference-time repair of an invalid or incomplete drifted call.

Our central question is whether an inference-time repair layer can recover the performance lost from Original to Drifted without harming examples that were already correct.

4 Method

4.1 SchemaShield-Lite

SchemaShield-Lite is a lightweight inference-time robustness layer with three parts. Figure 1 sketches the full pipeline.

Canonical tool card. Each tool is rendered into a compact canonical form containing the tool name, description, required fields, and field-level type information. This normalizes interface presentation across tools and emphasizes current field names.

Validation. After the base model emits a tool call, a validator checks tool name, required fields, and argument structure. Validation produces a structured error report rather than a binary failure signal.

One-shot repair. If the drifted call is invalid, the model receives a repair prompt containing the task context, canonical tool card, invalid call, and validation errors. In the primary non-oracle protocol, repair proceeds in two stages. If the base call names a visible tool, validation runs against that tool and a constrained repair call is issued for that same visible tool. If the base call omits the tool name, we first run a candidate-set retry over the visible tools to produce a concrete

tool call; once a visible tool is resolved, a second repair call may be constrained to that selected tool. Where provider support allows it, this second step uses *forced-tool repair*; otherwise we fall back to unconstrained tool calling or JSON repair.

As our component ablation in Section 6.5 shows, it is the combination of structured validation errors and tool-constrained repair that does the heavy lifting once a visible target tool has been resolved. The canonical card is useful at the initial prediction step as drift-prevention prompting, but contributes negligibly to repair-stage recovery in our oracle-constrained upper-bound ablations.

4.2 Repair Prompting Details

Three implementation details turned out to matter materially:

- The repair prompt explicitly instructs the model to infer all required fields from the dialogue even when the invalid call is empty.
- When the invalid call omits the tool name, a candidate-set retry over the visible tools is needed before tool-constrained repair can fire.
- The repair stage uses a larger token budget than the base prediction stage. Raising repair capacity from the earlier default to 512 tokens fixed truncated repair outputs on longer DICE dialogues.

4.3 Per-Example Evaluation Loop

Algorithm 1 runs each benchmark example through three stages (Original, Drifted, Repaired). Drift is injected at one point (`apply_drift`); the gold call is re-expressed in the renamed schema via `adapt_gold` before scoring; repair fires only when the validator flags the drifted call as invalid. The three per-example booleans are averaged over $\mathcal{B}$ to produce the headline Original, Drifted, and Repaired accuracies.

Algorithm 1. Per-example evaluation under interface drift.

```
def evaluate(B, model, delta, validator, repair):
    trace = []
    for user_context, tool_interface, gold_call in B:
        # Stage 1: Original
        original_call = model(user_context, tool_interface)
        is_original_correct = score(original_call, gold_call)

        # Stage 2: Drifted
        drifted_interface = apply_drift(tool_interface, delta)
        drifted_gold = adapt_gold(gold_call, drifted_interface)
        drifted_call = model(user_context, drifted_interface)
        is_drifted_correct = score(drifted_call, drifted_gold)

        # Stage 3: Repaired (only when validation fails)
```

```
        validation = validator(drifted_call, drifted_interface)
        if validation.is_valid:
            repaired_call = drifted_call
        else:
            repaired_call = repair(drifted_call, validation)
        is_repaired_correct = score(repaired_call, drifted_gold)

        trace.append((is_original_correct, is_drifted_correct,
    is_repaired_correct))
    return aggregate(trace)
```

5 Experimental Setup

5.1 Benchmarks and Data Slices

We evaluate on two benchmarks. For DICE-BENCH [4], we use a 300-example subset drawn from single-function round-1 dialogues (allowing duplicate functions to reach sufficient scale). For BFCL [20], we use a 200-example subset drawn in equal parts from four categories: simple Python, multiple-function, live-simple, and live-multiple.

5.2 Models

We evaluate four open instruction-tuned models accessed via OpenRouter:

- **Qwen3.5-9B**: our primary model.
- **Qwen3.5-35B-A3B**: larger MoE variant.
- **Llama-4-Scout**: Meta's 2025 open model.
- **Mistral-Small-3.2-24B**: recent Mistral dense model.

All experiments use temperature 0.0 for low-variance evaluation, though hosted inference may still exhibit provider-side nondeterminism.

5.3 Drift Configurations

For DICE main runs, we use *mild* severity: two description modes (legacy-example and marketing), two schema modes (rename-parameters and reorder-parameters), and one distractor tool. For BFCL we use *medium* severity: three description modes, three schema modes, and three distractors, reflecting a more aggressive drift scenario. These operators are author-designed and deployment-motivated rather than mined from real API changelogs; we return to that limitation in Section 8. To sanity-check ecological validity, we manually audited 11 official changelog items from Notion and Stripe spanning 2022–2026. Six map directly to our documentation/example and parameter/schema operators, three

more are close candidate-set analogues where vendors add or consolidate neighboring tools or endpoints, and two are larger structural refactors outside our semantics-preserving scope [9,16,17,19,18]. We therefore treat the synthetic operators as a compact model of common surface-form churn rather than as a measured distribution over all API evolution events.

5.4 Repair Protocols

We use two repair protocols. Our *primary* protocol is non-oracle and is the main evidence in this paper: the validator checks the model-selected visible tool, and when the model omits the tool name, a candidate-set retry over only the visible tools resolves a target before repair. For broader model coverage and component ablations, we also retain earlier *oracle-constrained upper-bound* sweeps in which the repair stage is given the benchmark target tool. These upper-bound runs remain useful for cross-model context and ablation, but they are not the paper's deployment-facing evidence.

5.5 Metrics

We report:

- **Original score**: accuracy on the unmodified interface.
- **Drifted score**: accuracy under interface drift.
- **Repaired score**: accuracy after one-shot repair.
- **Naive retry score**: accuracy under a second identical drifted call (no canonical card, no validation feedback). This baseline isolates the value of simply retrying.
- **Originally-correct slice**: the subset of examples that the model solves correctly before drift. This slice isolates *true drift harm* from baseline errors.
- **Repair recoveries**: drift-induced failures recovered by repair.
- **Repair harms**: drifted-correct examples broken by repair.
- **Repair trigger rate**: fraction of examples for which the validator invokes the repair step.

5.6 Reproducibility and Release

All reported results come from fixed benchmark slices, fixed drift configurations, and fixed random seeds. The per-run JSON artifacts store example IDs, prompts, drifted tool schemas, validation outputs, repair targets, repair strategy, and token usage, and our evaluation uses the normalized semantic matching policy reported in each run summary. To preserve double-blind review, the initial submission omits author-identifying artifact links; we will release the experiment code, drift-generation scripts, prompts, configs, subset definitions, scoring/validation logic, and raw run artifacts in the artifact package for camera-ready release. Appendix A documents four representative case studies so the paper's main success and limitation modes can be checked without searching through the full run outputs.

Table 1: Primary non-oracle Qwen3.5-9B results on DICE-300 and BFCL-200, with oracle upper bounds. Brackets give 95% bootstrap CIs (seed 42).

Benchmark	Protocol	Orig. [95% CI]	Drift. [95% CI]	Repair [95% CI]	OC	Rec/Miss	Harms	Trig%
DICE-300	Non-oracle + candidate retry	0.757 [0.70, 0.80]	0.717 [0.67, 0.77]	**0.813 [0.77, 0.86]**	227	19/22	0	16.0
	Oracle upper bound	0.750 [0.70, 0.80]	0.713 [0.66, 0.76]	**0.827 [0.79, 0.87]**	225	17/18	0	21.0
BFCL-200	Non-oracle + candidate retry	0.790 [0.74, 0.84]	0.580 [0.51, 0.65]	**0.805 [0.75, 0.86]**	158	33/45	0	32.0
	Oracle upper bound	0.790 [0.74, 0.84]	0.640 [0.58, 0.71]	**0.795 [0.74, 0.85]**	158	23/35	0	31.0

OC = originally-correct count; Rec = recoveries on drift-induced failures; Harms = repair harms on originally-correct slice; Trig% = repair trigger rate. Rows come from separate runs, so Original scores may differ slightly across protocols.

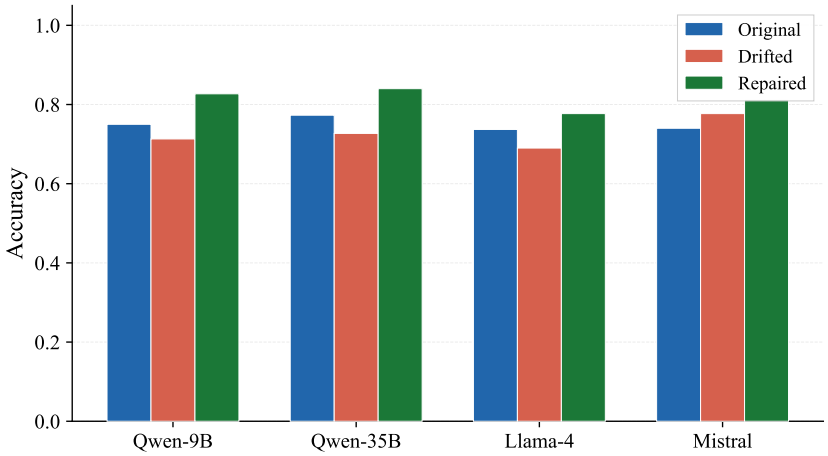

Fig. 2: Oracle-constrained upper-bound DICE sweep across four models. Primary evidence is the non-oracle protocol in Table 1.

6 Results

6.1 Primary Non-Oracle Results

Table 1 summarizes our primary non-oracle Qwen3.5-9B results on DICE-300 and BFCL-200, together with oracle-constrained upper bounds for reference only. Under the non-oracle candidate-retry protocol, SchemaShield-Lite improves over the drifted baseline on both benchmarks with no observed repair harms on the originally-correct slice.

The primary non-oracle results are strong on both benchmarks. On DICE-300, repair improves Qwen3.5-9B from 0.717 to 0.813 and recovers 19/22 drift-induced failures on the originally-correct slice. On BFCL-200, repair improves from 0.580 to 0.805 and recovers 33/45 drift-induced failures. The remaining gap to the oracle upper bound is small on DICE (0.813 vs 0.827), and on BFCL the final non-oracle candidate-retry run slightly exceeds the earlier oracle-style run (0.805 vs 0.795).

We therefore focus our interpretation on the large repaired-vs-drifted gaps, not on small differences between nearby runs. The bootstrap intervals in Table 1

support the main claim that structured repair helps, but they do not justify strong claims about minor oracle-vs-non-oracle differences. Paired exact McNemar tests on the final non-oracle runs also support the main repair effect: repaired beats drifted with $p = 3.7 \times 10^{-9}$ on DICE and $p = 5.7 \times 10^{-14}$ on BFCL. On shared examples, the repaired-score difference between the oracle upper bound and the final non-oracle protocol is itself not significant on either benchmark (DICE exact McNemar $p = 0.125$; BFCL $p = 0.754$), which is consistent with the small raw score gaps in Table 1.

The candidate-set retry is load-bearing in the non-oracle setting. Relative to an earlier non-oracle run that simply skipped unresolved tool names, candidate retry raises repaired accuracy from 0.780 to 0.813 on DICE and from 0.725 to 0.805 on BFCL, while reducing unresolved repair targets from 35 to 15 and from 34 to 9 respectively.

Oracle upper bounds for context. Figure 2 and the oracle rows in Table 1 remain useful as upper-bound context, but they are not the paper's main evidence. We use them to localize mechanisms and portability limits, not to define the deployment-facing claim.

Repair cost overhead. On the final non-oracle Qwen3.5-9B runs, SchemaShield-Lite triggers repair on 48/300 examples (16.0%) in DICE and 64/200 examples (32.0%) in BFCL. Total repair tokens are 103,221 against 316,185 drifted tokens on DICE (32.6% relative overhead) and 154,249 against 327,289 drifted tokens on BFCL (47.1% relative overhead). Because repair is gated by validation, this overhead is paid only when the drifted call is invalid or incomplete. On a refreshed BFCL-200 live rerun of the same non-oracle protocol with latency instrumentation, average original-call latency is 8.8s, average drifted-call latency is 9.3s, and average repair-call latency when repair fires is 13.0s. With a 27.0% repair trigger rate on that rerun, this corresponds to roughly 3.5s expected added latency per example.

6.2 Baselines Do Not Explain the Gains

A first-order alternative explanation is that repair works simply because the model gets a second attempt. We test this with two baselines (Table 2). *Naive retry* re-runs the identical drifted prompt with no new information. *Tool-list re-prompt* tells the model the call was invalid and lists the available tool names, but provides no canonical card, no validation errors, and no view of the invalid call. This second baseline isolates whether simply flagging the error and listing tools explains the gains.

Neither baseline approaches SchemaShield-Lite. Naive retry reaches 0.727 on DICE and 0.610 on BFCL; tool-list re-prompt improves only marginally to 0.753 and 0.625 respectively. Both remain far below SchemaShield-Lite (0.813 and 0.795). The structured validation feedback, not merely knowing which tools are available, is therefore the load-bearing component. The paired repaired-vs-naive contrast is highly significant on both benchmarks (exact McNemar $p = 4.1 \times 10^{-9}$ on DICE; $p = 1.1 \times 10^{-13}$ on BFCL).

Table 2: Baselines vs SchemaShield-Lite on Qwen3.5-9B (non-oracle). Scores from separate runs; minor differences from Table 1 reflect run-level variance.

Benchmark	Drifted	Naive Retry	Tool-List Re-prompt	SchemaShield-Lite	Δ over Drifted
DICE-300	0.713	0.727	0.753	**0.813**	+0.100
BFCL-200	0.590	0.610	0.625	**0.795**	+0.205

Table 3: Decomposition of repaired improvements on the primary non-oracle Qwen3.5-9B runs.

Benchmark	Total gains	Drift recovery	Baseline patching	Harms
DICE-300	29	19	10	0
BFCL-200	45	33	12	0

6.3 Repair Coverage: Drift Recovery Dominates and Most Misses Are Validator-Visible

The "repair > original" effect is real, but it does not dominate the main story. Table 3 decomposes every repaired improvement over the drifted call into either *drift recovery* on the originally-correct slice or *baseline patching* on examples the base model already missed before drift. Most repaired gains come from genuine drift recovery: 19/29 on DICE and 33/45 on BFCL. Baseline patching is material, especially on BFCL, and should be read as a secondary benefit rather than the paper's central causal claim.

These runs also reveal whether drift misses actually enter the repair path. On the originally-correct slice, 20/22 DICE drift misses and all 45/45 BFCL drift misses are validator-visible invalid or incomplete calls. Only two DICE misses remain validator-invisible valid-but-wrong outputs, and BFCL has none. Across the full benchmark slices, 63/85 DICE drift misses and 73/84 BFCL drift misses are validator-visible. The repair stack therefore addresses most observed drift failures; valid-but-wrong calls that bypass validation are a residual rather than dominant failure mode.

6.4 Drift-Type Ablation: Candidate-Set Drift Dominates

Table 4 decomposes the drift pipeline into its three components and evaluates each in isolation at DICE-300 on Qwen3.5-9B. Description drift alone causes only a 0.013 point accuracy drop. Schema drift alone causes only 0.007. By contrast, candidate-set drift (adding a distractor tool) causes a 0.047 drop, matching the full combined drift scenario (0.050). Because this ablation is about where the damage comes from, the main interpretation still rests on the Original-to-Drifted delta. For the repair column, however, only the distractor-bearing rows require fresh non-oracle evidence. In our DICE-300 slice, every baseline task has exactly one candidate tool, so under predicted-tool repair the description-only and

Table 4: Drift-type ablation on DICE-300 with Qwen3.5-9B. Single-candidate tasks make description/schema-only repair equivalent to oracle-target.

Drift type	Orig.	Drift.	Repair	Drift Drop
Description only	0.760	0.747	0.813	−0.013
Schema only	0.767	0.760	0.830	−0.007
Candidate-set only	0.757	0.710	0.817	−0.047
All combined	0.763	0.713	0.813	−0.050

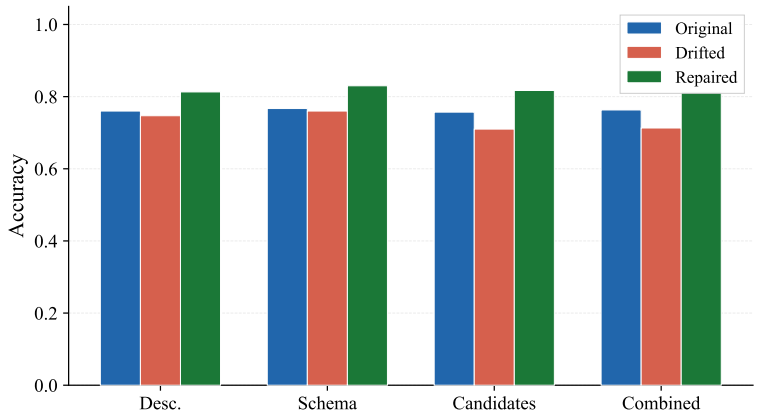

Fig. 3: Drift-type ablation on DICE-300 with Qwen3.5-9B. Distractor tools account for nearly all drift damage; description and schema drift are negligible.

schema-only settings collapse to the same effective repair target as the oracle-target protocol via single-candidate fallback. The non-oracle question therefore matters only once distractor tools are introduced. A fresh candidate-set-only non-oracle rerun reaches 0.817, while the primary DICE main run already supplies the matching all-combined non-oracle row at 0.813. The qualitative conclusion is unchanged: candidate-set drift remains the dominant source of damage, and non-oracle repair stays close to the earlier oracle upper bounds.

This result qualifies a common framing in the interface-drift literature. On this Qwen3.5-9B DICE-300 ablation, minor description rewording and semantics-preserving parameter renames do not meaningfully degrade accuracy; the model is evidently robust to this class of surface perturbation. The observed failure mode is *candidate-set confusion*: when a distractor tool with an overlapping description is added to the candidate set, tool selection degrades. In this slice, the practical implication is that tool-catalog growth deserves closer monitoring than minor description or schema revisions.

Table 5: Oracle-constrained upper-bound component ablation on DICE-300 and BFCL-200 with Qwen3.5-9B. BFCL uses medium-severity drift, DICE uses mild.

Benchmark	Variant	Drift.	Repair	Rec/Miss	Harms	Trig%
DICE-300	Canonical card only	0.770	0.770	0/9	0	0.0
	Validation + retry (no card)	0.730	0.830	16/17	0	20.0
	Full SchemaShield-Lite	0.713	**0.827**	17/18	0	21.0
BFCL-200	Canonical card only	0.810	0.810	0/11	0	0.0
	Validation + retry (no card)	0.645	**0.820**	26/34	0	29.5
	Full SchemaShield-Lite	0.640	0.795	23/35	0	31.0

6.5 Upper-Bound Component Ablation: Validation, Not Canonical Card, Drives Recovery

To isolate which parts of the repair stack matter once a target tool has already been resolved, Table 5 evaluates three variants of SchemaShield-Lite in an oracle-constrained upper-bound setting on both DICE-300 and BFCL-200 with Qwen3.5-9B: (i) canonical card only, where the card is prepended to the initial prompt but no validation/repair is applied; (ii) validation-plus-retry, which uses validation errors in a repair prompt but omits the canonical tool card from the repair context; and (iii) full SchemaShield-Lite (card + validation + repair).

Within this oracle-constrained setting, the BFCL rows confirm and *strengthen* the DICE finding in three ways. First, the canonical card at initial-prediction time provides even stronger drift prevention on BFCL than on DICE: the base drifted score of 0.640 jumps to 0.810, a 17.0-point drift-prevention effect. Second, the card-only configuration still never triggers the validator and recovers zero drift-induced failures, because its valid-but-imperfect outputs pass schema checking without entering the repair path. Third, validation-plus-retry without the canonical card at repair time reaches a *higher* repaired accuracy on BFCL than the full pipeline (0.820 vs 0.795, and 26/34 recoveries vs 23/35), matching the pattern observed on DICE where validation-only nearly ties the full pipeline.

Taken together across both benchmarks, these upper-bound numbers localize the load-bearing component of SchemaShield-Lite to the validation error feedback loop combined with tool-constrained repair, not the canonical tool card presented at repair time. The card earns its keep as *drift-prevention prompting* at the initial prediction step, where BFCL shows it can dominate the remaining drift damage, but can be dropped from the repair prompt without hurting recovery once the target tool has already been resolved. Because these ablations are oracle-constrained, we treat them as upper-bound mechanism evidence rather than as a full explanation of non-oracle portability.

6.6 Error Analysis

On the primary non-oracle Qwen3.5-9B runs, only 3 DICE examples and 12 BFCL examples remain unrecovered on the originally-correct slice. The DICE

Table 6: Unrecovered oracle-constrained DICE cases on non-primary models. "Artifact" = formatting/paraphrase mismatches, semantically correct on inspection.

Model	Unrecovered	Artifact-like	Genuine failure
Qwen3.5-35B-A3B	5	4	1
Llama-4-Scout	4	4	0
Mistral-24B	4	3	1
Total	13	11	2

residuals are split between one repaired-but-still-mismatched hashtag value and two validator-invisible value mismatches that never trigger repair. On BFCL, most unrecovered cases remain validator-visible missing-field failures after repair, while two cases remain unresolved because the model never supplies a usable tool name. These patterns are consistent with the main bottleneck story in Section 6.3: the dominant remaining failure modes are either missing-tool-name resolution or persistent argument completion failures.

Across the oracle-constrained DICE upper-bound sweeps, no repair harms are observed. However, not every drift miss is recovered. On Qwen3.5-35B, repair recovers 20 of 25 drift misses; on Llama-4-Scout, 12 of 16; on Mistral, 8 of 12. We manually inspected every unrecovered case on these three runs and classified them against the gold call.

The pattern is consistent across models: nearly all unrecovered examples involve the repair call selecting the correct tool with well-formed arguments whose string values disagree with gold only by formatting or paraphrase. On Qwen3.5-35B (5 unrecovered), these include one strict whitespace/punctuation difference ("`the quick brown fox...`" vs "`The quick brown fox....`") and three enum-format or paraphrase-level mismatches (e.g. `regular_season` vs `regular season`, `the office` vs `Office`); a single genuine failure has an empty arguments object after repair. On Llama-4-Scout (4 unrecovered), all four are format-level: `ISO`-prefixed dates vs two-field dates (e.g. `2023-11-10` vs `11-10`), or JSON with quoted-number string values rather than numeric literals. On Mistral-24B (4 unrecovered), the misses include near-synonym topics (`inspirational` vs `inspiration`, `AI` vs `artificial intelligence`, `Google` vs `GOOGL`) and one wrong numeric amount. Across 13 unrecovered cases in total, only 2 are genuine inference failures (empty arguments and one wrong value); the remaining 11 are strict-match artifacts rather than true drift failures. The practical implication is that the paper's reported recovery rates are a lower bound: a lenient evaluator would count most of the unrecovered cases as correct.

Mistral-24B exhibits an unusual pattern: its drifted score (0.777) slightly exceeds its original score (0.740). Inspection shows that 15/23 apparent drift "gains" are strict-match artifacts on the original call (formatting differences in string values) rather than information recovered from the drifted description. We read this as scorer noise, not evidence that drift helps.

6.7 Additional Non-Oracle Replications

To reduce the single-model dependence of the primary story, we also ran the same BFCL-200 non-oracle candidate-retry protocol on Qwen3.5-35B-A3B. Repair again helps materially, lifting accuracy from 0.635 to 0.845 (exact McNemar $p = 4.5 \times 10^{-13}$). On the originally-correct slice it recovers 33/38 drift misses, shows no observed repair harms, and leaves only one unresolved repair target. This does not eliminate same-family dependence, but it shows that the main BFCL non-oracle effect is not confined to the 9B run.

We also ran the same BFCL-200 non-oracle candidate-retry protocol on Llama-4-Scout. Repair still improves over the drifted call ($0.635 \rightarrow 0.665$, exact McNemar $p = 0.031$) and again shows no observed repair harms on the originally-correct slice. However, this run exposes an important portability failure mode: the model omits the tool name frequently, leaving 43 unresolved repair targets and limiting recovery on the originally-correct slice to just 1/11 drift misses. The Llama run therefore supports some portability of the repair effect, but also reinforces that non-oracle repair quality depends heavily on the base model's ability to emit a usable tool name.

We also ran a BFCL-200 non-oracle sanity check on GPT-4o-mini after adding provider-safe tool-name aliasing for OpenAI-style tool schemas. On this stronger closed model, drift damage is much smaller to begin with: accuracy falls only from 0.865 to 0.840. Repair lifts the score to 0.860 with no observed harms and zero unresolved repair targets, but the gain is small and not significant on this 200-example slice (exact McNemar $p = 0.125$). We therefore read the GPT-4o-mini result as a credibility check rather than as a new headline number: the problem appears less severe on a stronger closed model, yet the same non-oracle repair stack does not appear to hurt it.

7 Discussion

Our results support three narrower conclusions than the original SchemaShield-Lite framing suggested. First, non-oracle inference-time repair is a viable robustness layer for tool-calling systems under interface drift: on Qwen3.5-9B it improves over drifted performance on both DICE and BFCL while showing no observed repair harms on the originally-correct slice. Second, the main non-oracle bottleneck is missing tool names; a candidate-set retry over the visible tools closes much of the gap to the oracle upper bound. Third, on our DICE-300 primary-model ablation, candidate-set drift is a more important failure mode than description or schema drift.

Together these findings simplify the deployment picture. A practitioner does not need a heavy per-tool canonicalization pipeline; a visible-tool validator, a candidate-set retry for missing tool names, and a constrained repair call once a tool is resolved already recover most of the damage we observe. And the monitoring story is clearer: in our slices, tool-catalog expansion appears more consequential than minor description or parameter rename churn. The additional

BFCL-200 Qwen3.5-35B-A3B replication suggests this practical recipe is not limited to one exact model size, even if broader cross-family evidence remains thin.

These same results also matter for adversarial interface manipulation. Faghih et al. [1] show that small description edits can steer tool selection even when semantics are unchanged. Our ablation suggests that the more serious surface to monitor is not just description wording in isolation, but the combination of description overlap and candidate-set expansion: once distractor tools enter the visible set, routing degrades quickly. In that sense, the non-oracle validator-plus-repair stack is relevant not only to benign interface drift but also to description-surface attacks that aim to induce tool confusion.

8 Limitations

Several limitations remain. First, although we now include a strong BFCL non-oracle replication on Qwen3.5-35B-A3B and a weaker portability check on Llama-4-Scout, the cleanest successful non-oracle evidence is still concentrated in the Qwen3.5 family; the broader multi-model results retained in the paper remain mostly oracle-constrained upper bounds rather than full non-oracle replications. Second, the BFCL evaluation now covers three open-model non-oracle runs, but two are Qwen3.5-family results and the third is materially bottlenecked by missing tool names; we also include one closed-model BFCL sanity check, but not yet a broader closed-model sweep. Third, our drift operators focus on stale descriptions, renamed fields, and distractors; other forms of interface drift (for example, type narrowing, enum replacement, or multi-tool refactors) remain untested, and although we now add a small manual audit of official vendor changelogs, the perturbations are still author-designed rather than mined from a larger empirical change distribution. Fourth, the drift-type and component ablations remain oracle-constrained upper bounds, so their mechanism-level conclusions may not fully transfer when non-oracle tool resolution fails. Fifth, evaluation uses hosted inference and so may incorporate provider-specific tool-calling behavior. Sixth, the "repair > original" effect indicates that SchemaShield-Lite is also patching baseline failures, useful in deployment, but a complication for clean attribution to drift recovery specifically. Seventh, strict string-matching against gold calls undercounts true recoveries: manual inspection of the 13 unrecovered cases on the three non-primary models (Section 6.6) shows that 11 are formatting or paraphrase artifacts, so the reported recovery rates should be read as lower bounds. Finally, while we now report broader cross-protocol significance for oracle-vs-non-oracle repaired outcomes on the shared Qwen slices, we still lack a broader wall-clock latency sweep across benchmarks, providers, and models.

9 Conclusion

We studied SchemaShield-Lite, a lightweight inference-time validation-and-repair stack for tool calling under interface drift. On Qwen3.5-9B under a non-oracle

candidate-retry protocol, SchemaShield-Lite improves DICE-300 from 0.717 to 0.813 and BFCL-200 from 0.580 to 0.805, with no observed repair harms on the originally-correct slice in both benchmarks. Three findings qualify the picture. A naive-retry baseline does not explain the gains. Description and schema drift cause only small degradations on our DICE slice, while candidate-set drift is the dominant failure mode. And in the non-oracle setting, missing tool names are the main repair bottleneck: a candidate-set retry over the visible tools substantially reduces unresolved cases and closes most of the gap to oracle-constrained upper bounds. Additional oracle-constrained sweeps across other models and upper-bound ablations remain useful context, and a BFCL sanity check on GPT-4o-mini suggests that stronger closed models suffer less drift damage to begin with. A same-protocol BFCL-200 replication on Qwen3.5-35B-A3B (0.635 $\rightarrow$ 0.845) materially strengthens the main claim, but the cleanest successful non-oracle evidence is still concentrated in the Qwen family. The paper should therefore be read as a controlled study of drift sensitivity and inference-time repair rather than as a complete account of production interface drift.

Broader impact. The same repair stack that recovers benign drift could in principle mask adversarial tool injection, where a malicious distractor tool is added to the candidate set. Practitioners deploying inference-time repair should pair it with provenance checks on newly added tools.

GenAI disclosure. Generative AI tools (Claude, ChatGPT) were used during manuscript preparation for drafting assistance and code review. All AI-generated content was reviewed, edited, and verified by the authors, who take full responsibility for the final text.

References

1. Faghih, K., Wang, W., Cheng, Y., Bharti, S., Sriramanan, G., Balasubramanian, S., Hosseini, P., Feizi, S.: Gaming tool preferences in agentic llms. arXiv preprint arXiv:2505.18135 (2025)
2. Geng, S., Josifoski, M., Peyrard, M., West, R.: Grammar-constrained decoding for structured NLP tasks without finetuning. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (2023)
3. Guo, Y., et al.: Learning to rewrite tool descriptions for reliable llm-agent tool use. arXiv preprint arXiv:2602.20426 (2026)
4. Jang, K., Lee, D., Kim, K., Heo, D., Lee, T., Kim, W., Suh, B.: Dice-bench: Evaluating the tool-use capabilities of large language models in multi-round, multi-party dialogues. arXiv preprint arXiv:2506.22853 (2025)
5. Lercher, A., Bauer, C., Macho, C., Pinzger, M.: Autoguard: Reporting breaking changes of rest apis from java spring boot source code. In: 2025 IEEE 32nd International Conference on Software Analysis, Evolution and Reengineering (SANER) (2025)
6. Li, M., Zhao, Y., Yu, B., Song, F., Li, H., Yu, H., Li, Z., Huang, F., Li, Y.: API-bank: A comprehensive benchmark for tool-augmented LLMs. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. pp. 3102–3116 (2023)

7. Liu, Z., Hoang, T., Zhang, J., Zhu, M., Lan, T., Kokane, S., Tan, J., Yao, W., Liu, Z., Feng, Y., Murthy, R., Yang, L., Savarese, S., Niebles, J.C., Wang, H., Heinecke, S., Xiong, C.: Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets. arXiv preprint arXiv:2406.18518 (2024)
8. Lu, J., Holleis, T., Zhang, Y., Aumayer, B., Nan, F., Bai, F., Ma, S., Ma, S., Li, M., Yin, G., Wang, Z., Pang, R.: Toolsandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. arXiv preprint arXiv:2408.04682 (2024)
9. Notion: Changelog. https://developers.notion.com/page/changelog (2026), notion developer changelog, accessed 2026-04-14
10. Park, K., Wang, J., Berg-Kirkpatrick, T., Polikarpova, N., D'Antoni, L.: Grammar-aligned decoding. In: Advances in Neural Information Processing Systems (2024)
11. Patil, S.G., Zhang, T., Wang, X., Gonzalez, J.E.: Gorilla: Large language model connected with massive apis. arXiv preprint arXiv:2305.15334 (2023)
12. Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z., Sun, M.: Toolllm: Facilitating large language models to master 16000+ real-world apis. arXiv preprint arXiv:2307.16789 (2023)
13. Rabinovich, E., Anaby-Tavor, A.: On the robustness of agentic function calling. In: Proceedings of the 5th Workshop on Trustworthy NLP (2025)
14. Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language models can teach themselves to use tools. arXiv preprint arXiv:2302.04761 (2023)
15. Serbout, S., Pautasso, C.: An empirical study of web api versioning practices. In: Web Engineering. Lecture Notes in Computer Science (2023)
16. Stripe: Replaces line item parameters from the create checkout session endpoint. https://docs.stripe.com/changelog/2022-08-01/replaces-line-item-create-checkout-session (2022), stripe changelog entry, accessed 2026-04-13
17. Stripe: Adds default value to custom fields on payment links. https://docs.stripe.com/changelog/basil/2025-03-31/default-value-payment-links-custom-fields (2025), stripe changelog entry, accessed 2026-04-13
18. Stripe: Renames the parameter for custom settlement timing on the balance settings api. https://docs.stripe.com/changelog/basil/2025-07-30/balance-settings-delay-days-rename (2025), stripe changelog entry, accessed 2026-04-14
19. Stripe: Replaces upcoming invoice api methods with the create preview invoice api. https://docs.stripe.com/changelog/basil/2025-03-31/invoice-preview-api-deprecations (2025), stripe changelog entry, accessed 2026-04-14
20. Yan, F., Mao, H., Ji, C.C.J., Zhang, T., Patil, S.G., Stoica, I., Gonzalez, J.E.: Berkeley function calling leaderboard. https://gorilla.cs.berkeley.edu/leaderboard.html (2024)
21. Yao, S., Shinn, N., Razavi, P., Narasimhan, K.: τ-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv preprint arXiv:2406.12045 (2024)
22. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: React: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629 (2022)
23. Zhang, Y., et al.: Robust tool use via fission-grpo. arXiv preprint arXiv:2601.15625 (2026)

24. Zhong, Y., et al.: Complexfuncbench: Exploring multi-function calling in large language models. arXiv preprint arXiv:2501.10132 (2025)

A Worked Appendix Cases

This appendix is intended to be readable on its own. A reviewer should be able to understand the examples below without opening the raw JSON artifacts. The full artifact trail is therefore optional and will be included in the artifact package for camera-ready release.

A.1 How to read the case studies

Each case study uses the same five fields from the stored run artifacts.

- `original_call`: the model's output on the clean interface before any drift is applied.
- `pred_call`: the model's first output on the drifted interface, before repair.
- `validation`: the structured error report produced by the validator on the drifted prediction.
- `repaired_call`: the output after the repair stack runs.
- `drifted_gold_call` and `repaired_match`: the benchmark's expected answer under the renamed drifted schema, and whether the repaired output matches it.

The repair strategies mentioned below have a simple interpretation. `candidate_tool_retry` is used when the drifted prediction does not provide a usable tool name, so the system must first recover a tool from the visible candidate set. `forced_tool_call` is used when the tool name is already known, allowing the repair stage to stay constrained to that tool.

A.2 Case 1: DICE missing-tool-name failure under candidate-set drift

This is the clearest instance of the paper's main non-oracle bottleneck. The task is a multi-turn conversation in which the user gradually specifies a haircut appointment with four required fields: service, date, time, and location. On the clean interface, the tool is `schedule_appointment(service, date, time, location)`, and the base model gets the example exactly right.

```
{
  "name": "schedule_appointment",
  "arguments": {
    "service": "haircut",
    "date": "11-10",
    "time": "10:00",
    "location": "456 Salon Ave."
  }
}
```

Drift then changes the interface in two ways. First, the schema renames `date`, `time`, and `location` to `date_v2`, `time_v2`, and `location_v2`. Second, the candidate set expands to include a distractor tool, `schedule_appointment_alt_1`, while the description includes a legacy note that still mentions the older payload style. Under this drifted interface, the extracted prediction is not merely wrong; it is empty:

```
{
  "name": "",
  "arguments": {}
}
```

That empty prediction should be read carefully. It does not mean the benchmark gold answer was empty. It means the runtime could not extract any usable tool call from the provider response. In the implementation, the extraction routine in openrouter_client.py returns this exact object when the response contains neither a usable `tool_calls` structure nor parseable JSON fallback. The validator therefore records the failure as `missing_tool_name`.

Once the validator sees that the tool name is missing, the repair stack switches to `candidate_tool_retry`. The repair prompt lists the two visible tools, asks the model to choose exactly one, and tells it to infer the required arguments from the dialogue context. That retry succeeds. The repaired call recovers the correct tool and the renamed argument fields:

```
{
  "name": "schedule_appointment",
  "arguments": {
    "service": "haircut",
    "date_v2": "11-10",
    "time_v2": "10:00",
    "location_v2": "456 Salon Ave."
  }
}
```

The repaired output matches the drifted gold call exactly. This is the core failure-and-recovery story of the paper in one example: the dominant non-oracle problem is often not a slightly malformed argument list, but complete loss of the tool name under candidate-set drift. Optional artifact trail: DICE JSON artifact at `L7423`, `L7508`, `L7621`, and `L7625`; parser code at openrouter_client.py `L112`.

A.3 Case 2: DICE forced-tool repair after partial compliance

This second example shows a different and easier failure mode. The task is again multi-turn DICE dialogue, this time asking for the top MLB pitchers by ERA in the 2023 regular season. On the clean interface the relevant tool is `pitching_leaders`, with fields `results`, `sort_column`, `season`, `sports_code`, and `game_type`; the model is correct before drift.

Under drift, the tool name stays the same but most required fields are renamed to `sort_column_v2`, `season_v2`, `sports_code_v2`, and `game_type_v2`. The field `results` is left unchanged, and the candidate set grows by one distractor tool, `pitching_leaders_alt_1`. On this drifted interface the model still names the correct tool, but it collapses on the arguments:

```
{
  "name": "pitching_leaders",
  "arguments": {}
}
```

This is important because it means candidate resolution is no longer the hard part. The validator can already resolve the intended tool and simply reports a set of `missing_field` errors for the required renamed fields. Because the tool target is already known, the system does not need `candidate_tool_retry`; it can go directly to `forced_tool_call` repair, keeping the repair stage constrained to `pitching_leaders`.

That constrained repair succeeds in one step. It reconstructs the full drifted call, including the renamed fields and the values implied by the conversation:

```
{
  "name": "pitching_leaders",
  "arguments": {
    "results": 10,
    "sort_column_v2": "ERA",
    "season_v2": "2023",
    "sports_code_v2": "MLB",
    "game_type_v2": "regular season"
  }
}
```

The repaired call matches the drifted gold call exactly. This example isolates a main mechanism claim of the paper: once the visible target tool has been resolved, the load-bearing part of the method is the validator error report plus tool-constrained repair, not tool search. Optional artifact trail: DICE JSON artifact at `L1816`, `L1913`, `L1949`, `L2060`, and `L2064`.

A.4 Case 3: DICE limitation when validation sees no structural error

This example is included to make the boundary of the method explicit. The task is simple: play `Bohemian Rhapsody` on a `JBL Speaker`. The clean tool is `play_music(music_name, device_name)`. Under drift, `device_name` is renamed to `device_name_v2`, the description gains a legacy note that still mentions the old field name, and a distractor `play_music_alt_1` is added.

The drifted prediction looks structurally fine:

```
{
  "name": "play_music",
  "arguments": {
    "music_name": "Bohemian Rhapsody",
    "device_name_v2": "JBL"
  }
}
```

The drifted gold call, however, expects `"JBL Speaker"` rather than `"JBL"`. That difference is semantically important under the benchmark's strict matching policy, but it is not a structural error. The tool name is correct. All required fields are present. The field names match the drifted schema. As a result, the validator marks the call `valid = true`, the repair strategy is `not_needed`, and the repaired call is identical to the drifted prediction.

This limitation matters because it prevents overclaiming. SchemaShield-Lite is a structure-level repair layer. It is designed to catch missing tool names, missing required fields, and schema-level incompatibilities. It is not a general semantic correction oracle that can decide when a shorter device description like `"JBL"` should be expanded to `"JBL Speaker"`. The example therefore shows exactly where the method stops helping: once a call passes structural validation, value-level semantic mismatches can survive unchanged. Optional artifact trail: DICE JSON artifact at `L38623`, `L38684`, `L38694`, `L38720`, and `L38741`.

A.5 Case 4: BFCL replication under heavier distractor pressure

The final worked example shows that the same pattern is not confined to DICE. In BFCL, the task is single-turn rather than conversational: compute displacement from initial velocity 10, acceleration 9.8, and time 5. On the clean interface the tool is `calculate_displacement`, with fields `initial_velocity`, `time`, and `acceleration`.

Under the medium-severity BFCL drift setting, the interface becomes noticeably messier than in the DICE examples. Parameter renaming composes across multiple steps, yielding the field names `initial_velocity_v2`, `time_v2_v2`, and `acceleration_v2_v2`. At the same time, the visible tool set expands from one tool to four: `calculate_displacement` plus three distractors `_alt_1`, `_alt_2`, and `_alt_3`. Under this heavier candidate pressure, the drifted prediction again collapses to:

```
{
  "name": "",
  "arguments": {}
}
```

Validation therefore reports `missing_tool_name` against the four-way candidate set. The repair stack then invokes `candidate_tool_retry`, which first resolves the visible tool and then reconstructs the required arguments from the task:

```
{
  "name": "calculate_displacement",
  "arguments": {
    "initial_velocity_v2": 10,
    "time_v2_v2": 5,
    "acceleration_v2_v2": 9.8
  }
}
```

The repaired call matches the drifted gold call exactly. This BFCL case matters because it shows that the DICE result is not only a conversational artifact. The same non-oracle bottleneck, missing tool names under distractor-heavy candidate sets, appears in a single-turn API-style benchmark as well. Optional artifact trail: BFCL JSON artifact at `L2875`, `L2938`, `L2952`, `L2968`, and `L3064`.

A.6 What the four cases collectively show

Taken together, these worked examples make the paper's claims easier to inspect. Case 1 shows the dominant non-oracle failure mode: the model can lose the tool name entirely once distractor tools are introduced. Case 2 shows that when the tool name survives, structured validation plus constrained repair can often recover every renamed field in one step. Case 3 shows the main limitation: structural validation cannot catch every value-level semantic mismatch. Case 4 shows that the same recovery pattern appears outside DICE, under BFCL's heavier candidate-set drift. In short, the appendix evidence supports the paper's central argument that interface drift is real, that missing tool names are the main deployment bottleneck, and that a lightweight validation-and-repair layer recovers a substantial part of the resulting damage without solving all semantic errors.